

MODULE 7 · MULTI-TENANCY & SECURITY

RBAC

Who can do what, where — Role-Based Access Control

Every API request is authenticated, then authorized

The API server checks two things on every call: **who are you** (authentication), then **are you allowed to do this** (authorization). RBAC is the authorization layer almost every cluster runs.

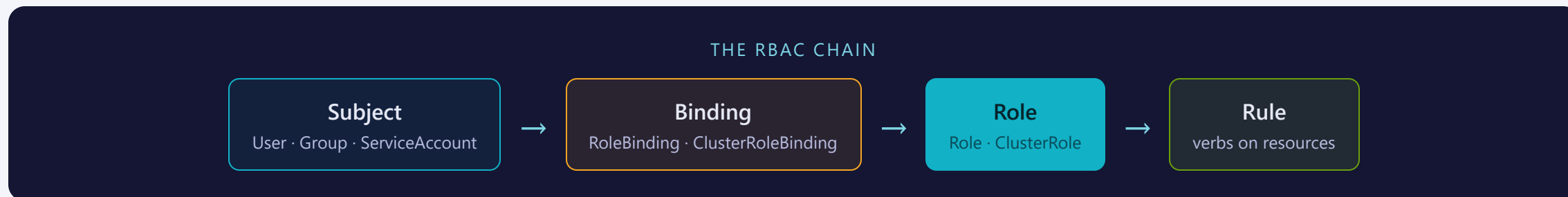
RBAC is purely additive and **deny-by-default** — a subject can do nothing until a rule explicitly grants it. That's exactly what a multi-tenant cluster needs: least privilege by default.

BY THE END YOU CAN

- Explain subject → binding → Role → rule
- Write a namespaced Role + RoleBinding
- Grant cluster-scope access with ClusterRole
- Test grants with `kubectl auth can-i`

CONCEPT

Subjects get rules only through a binding



A **Role** or **ClusterRole** is just a list of rules — verbs (get, list, watch, create, update, delete) against resources (pods, deployments, nodes...). On its own it grants nothing: only a **binding** that attaches it to a subject turns it into an actual permission.

Same rule shape, different reach

	NAMESPACED SCOPE	CLUSTER-WIDE SCOPE
Holds the rules	Role	ClusterRole
Binds subject → role	RoleBinding	ClusterRoleBinding

MENTAL MODEL

A RoleBinding can reference either a Role or a ClusterRole. Referencing a ClusterRole from a RoleBinding reuses one cluster-wide rule set (e.g. a shared "view" role) but scopes the grant to a single namespace — no full cluster access needed.

A Role's rules, granted to a subject via RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader-role
  namespace: rbac-lab
rules:
- apiGroups:
  - ""
  resources:
  - pods
  verbs:
  - get
  - list
  - watch
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: rbac-lab
subjects:
- kind: ServiceAccount
  name: pod-reader
  namespace: rbac-lab
roleRef:
  kind: Role
  name: pod-reader-role
  apiGroup: rbac.authorization.k8s.io
```

apiGroups ("" = core group), **resources**, and **verbs** are each lists. The RoleBinding's `roleRef` points at the Role by name; its `subjects` list says who gets it — both must live in the same namespace.

Imperative for a quick grant, declarative to keep it

IMPERATIVE — FAST, ONE-OFF

```
kubectl create role pod-reader-role \  
--verb=get,list,watch --resource=pods
```

```
kubectl create rolebinding pod-reader-binding \  
--role=pod-reader-role --serviceaccount=ns:sa
```

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f role.yaml
```

```
kubectl apply -f rolebinding.yaml
```

THE BRIDGE TRICK

Generate YAML from an imperative command, then edit & keep it:

```
$ kubectl create role pod-reader-role --verb=get,list,watch \  
--resource=pods --dry-run=client -o yaml > role.yaml
```

SEE IT

kubectl auth can-i answers before you try

`kubectl auth can-i VERB RESOURCE` returns yes or no without touching a real object. Add `--as=` to check as another subject — a User, Group, or ServiceAccount — with no context switch needed.

`--list` dumps every rule the subject's bindings resolve to, in one shot.

```
$ kubectl auth can-i list pods -n rbac-lab \  
    --as=system:serviceaccount:rbac-lab:pod-reader  
yes  
  
$ kubectl auth can-i delete pods -n rbac-lab \  
    --as=system:serviceaccount:rbac-lab:pod-reader  
no
```

THE PUNCHLINE

No matching rule means forbidden — always

RBAC has no "deny" rule. There's only grants, and the absence of one. If none of a subject's bound Roles or ClusterRoles contain a matching rule, the API server returns **403 Forbidden**.

A brand-new ServiceAccount starts with **zero** permissions — not "some default access." You opt in, resource by resource, verb by verb.



Where people trip

- **Wrong namespace.** A Role/RoleBinding is created wherever `kubectl` is currently pointed — easy to land it in the wrong namespace.
- **apiGroups: [""]** **confusion.** That's the core group (pods, services) — not "core" or a version string.
- **Role referenced across namespaces.** A RoleBinding can only bind a Role from its own namespace — cross-namespace needs a ClusterRole.
- **Testing as yourself.** `can-i` without `--as=` answers for your identity, not the ServiceAccount you're debugging.

Commands to keep at hand

```
$ kubectl create role NAME --verb=V --resource=R -n NS      # imperative Role
$ kubectl create rolebinding NAME --role=R --serviceaccount=NS:SA
$ kubectl auth can-i VERB RESOURCE -n NS --as=SUBJECT      # yes/no check
$ kubectl auth can-i --list --as=SUBJECT -n NS             # full rule set
$ kubectl describe role,rolebinding NAME -n NS            # inspect rules
```

Tip: `can-i --list` before you debug a "forbidden" — it shows exactly what a subject already has.

RECAP

RBAC in one breath

- Role/ClusterRole hold rules; RoleBinding/ClusterRoleBinding attach them to subjects
- Role = namespaced; ClusterRole = cluster-wide, or reusable from a RoleBinding
- No matching rule = forbidden — additive only, deny-by-default
- `kubectl auth can-i` tests grants without touching real objects

HANDS-ON

Now run Lab 7.2
[labs/7.2/](#)

Next: [7.3 — ServiceAccounts](#)